Máquinas de estados finitos

Prática

Prof. Roberto de Matos

roberto.matos@ifsc.edu.br

INSTITUTO FEDERAL
Santa Catarina
Câmpus São José

Objetivo

- Descrição VHDL de uma FSM
- Atribuição de estado
- *Buffer* de saída *Moore*

Objetivo

- Descrição VHDL de uma FSM
- Atribuição de estado
- *Buffer* de saída *Moore*

Leitura Recomendada:

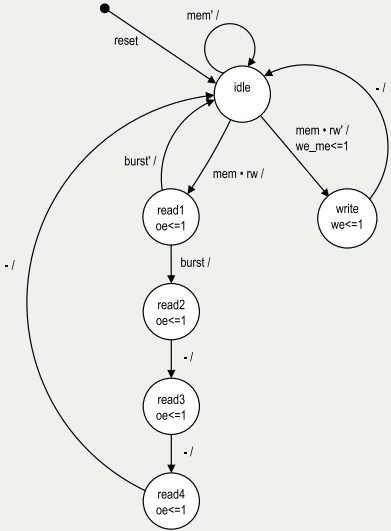
Capítulo 10 do livro: *RTL Hardware Design Using VHDL*

Descrição VHDL de uma FSM

Descrição VHDL de uma FSM

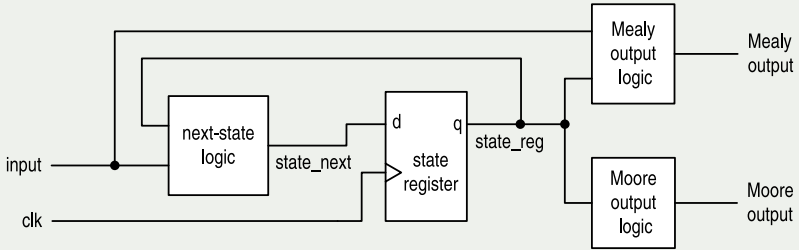
- Siga o diagrama de blocos básico
- Codifique a lógica do próximo estado e de saída de acordo com o diagrama de estado/ASM
- Use tipo de dados enumerado para estados

Descrição VHDL de uma FSM



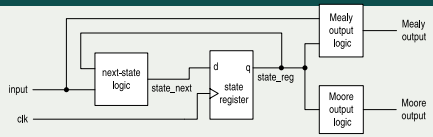
FSM - Codificação multi-segmento

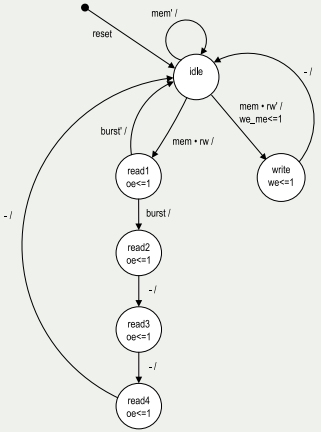
- Estilo de codificação multi-segmento:
  - 4 blocos no diagrama de blocos
  - 4 segmentos de código em VHDL
  - *Listing 10.1*: Arquitetura `mult_seg_arch` em list_10_01_to_06_memctrl.vhd



Listing 10.1 - Arquitetura `mult_seg_arch`

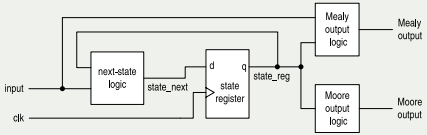
```
1  --------------------------------
2  -- Listing 10.1 memory controller FSM
3  --------------------------------
4  library ieee;
5  use ieee.std_logic_1164.all;
6  entity mem_ctrl is
7    port(
8      clk, reset: in std_logic;
9      mem, rw, burst: in std_logic;
10     oe, we, we_me: out std_logic
11   );
12 end mem_ctrl ;
13
14 architecture mult_seg_arch of mem_ctrl is
15   type mc_state_type is
16     (idle, read1, read2, read3, read4, write);
17   signal state_reg, state_next: mc_state_type;
18 begin
19   -- state register
20   process(clk,reset)
21   begin
22     if (reset='1') then
23       state_reg <= idle;
24     elsif (clk'event and clk='1') then
25       state_reg <= state_next;
26     end if;
27   end process;
```

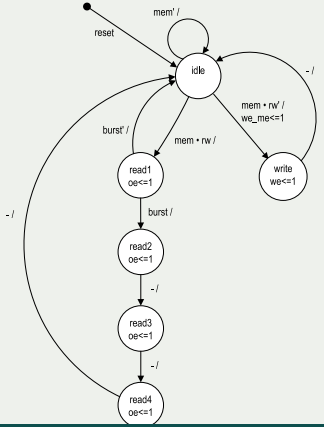




Listing 10.1 - Arquitetura `mult_seg_arch`

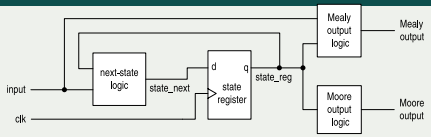
```
28  -- next-state logic
29  process(state_reg,mem,rw,burst)
30  begin
31    case state_reg is
32      when idle =>
33        if mem='1' then
34          if rw='1' then
35            state_next <= read1;
36          else
37            state_next <= write;
38          end if;
39        else
40          state_next <= idle;
41        end if;
42      when write =>
43        state_next <= idle;
44      when read1 =>
45        if (burst='1') then
46          state_next <= read2;
47        else
48          state_next <= idle;
49        end if;
50      when read2 =>
51        state_next <= read3;
52      when read3 =>
53        state_next <= read4;
54      when read4 =>
55        state_next <= idle;
56    end case;
57  end process;
```

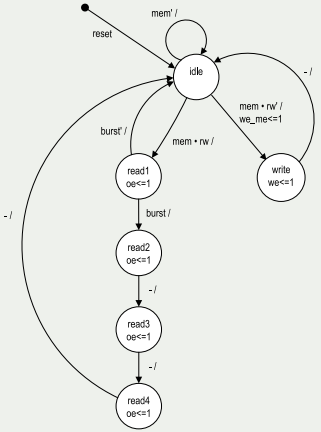




Listing 10.1 - Arquitetura `mult_seg_arch`

```
58 -- Moore output logic
59 process(state_reg)
60 begin
61   we <= '0'; -- default value
62   oe <= '0';  -- default value
63   case state_reg is
64     when idle =>
65     when write =>
66       we <= '1';
67     when read1 =>
68       oe <= '1';
69     when read2 =>
70       oe <= '1';
71     when read3 =>
72       oe <= '1';
73     when read4 =>
74       oe <= '1';
75   end case;
76 end process;
```



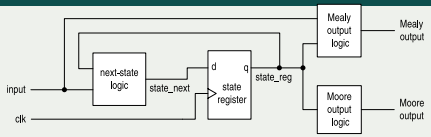


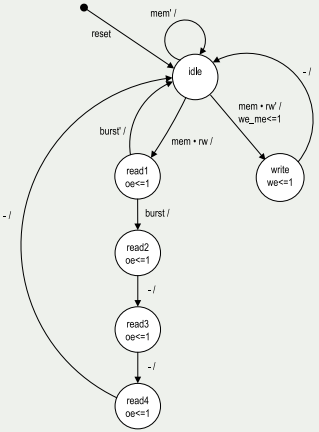
Listing 10.1 - Arquitetura `mult_seg_arch`

```
77    -- Mealy output logic
78    process(state_reg,mem,rw)
79    begin
80      we_me <= '0'; -- default value
81      case state_reg is
82        when idle =>
83          if (mem='1') and (rw='0') then
84            we_me <= '1';
85          end if;
86        when write =>
87        when read1 =>
88        when read2 =>
89        when read3 =>
90        when read4 =>
91      end case;
92    end process;
93  end mult_seg_arch;
```

Ou, simplesmente:

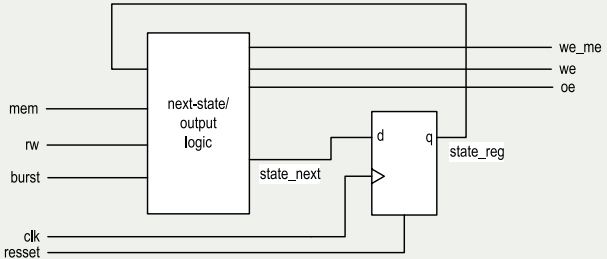
```
77  we_me <= '1' when ((state_reg=idle) and
78                     (mem='1') and
79                     (rw='0') ) else
80           '0';
```





FSM - Codificação com dois segmentos

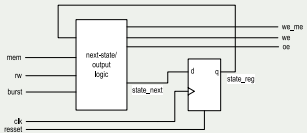
- Um segmento para o registrador
- Um segmento para o circuito combinacional:
  - Combinando as lógicas do próximo estado e das saídas *Moore* e *Mealy* juntos.
- *Listing 10.2*: Arquitetura `two_seg_arch` em list_10_01_to_06_memctrl.vhd

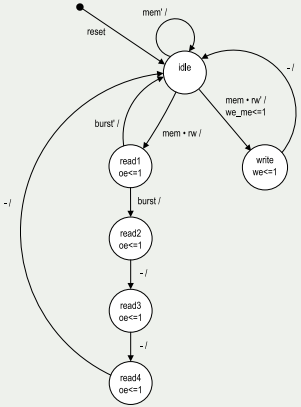


Listing 10.2 - Arquitetura `two_seg_arch` (lógica combinada)

```
113 -- next-state logic and output logic
114 process(state_reg,mem,rw,burst)
115 begin
116   oe <= '0';     -- default values
117   we <= '0';
118   we_me <= '0';
119   case state_reg is
120     when idle =>
121       if mem='1' then
122         if rw='1' then
123           state_next <= read1;
124         else
125           state_next <= write;
126           we_me <= '1';
127         end if;
128       else
129         state_next <= idle;
130       end if;
131     when write =>
132       state_next <= idle;
133       we <= '1';
134     when read1 =>
135       if (burst='1') then
136         state_next <= read2;
137       else
138         state_next <= idle;
139       end if;
140       oe <= '1';
```

```
141     when read2 =>
142       state_next <= read3;
143       oe <= '1';
144     when read3 =>
145       state_next <= read4;
146       oe <= '1';
147     when read4 =>
148       state_next <= idle;
149       oe <= '1';
150   end case;
151 end process;
```





Atribuição de Estado

Atribuição de Estado

- Atribua representações binárias a estados simbólicos.
- Em uma FSM síncrona:
  - Todas as atribuições funcionam
  - Uma boa atribuição reduz a complexidade da lógica do próximo estado/saída
- Atribuições típicas:
  - Binário, *gray-code*, *one-hot*, quase *one-hot*
- Exemplo:

	Binary assignment	Gray code assignment	One-hot assignment	Almost one-hot assignment
idle	000	000	000001	00000
read1	001	001	000010	00001
read2	010	011	000100	00010
read3	011	010	001000	00100
read4	100	110	010000	01000
write	101	111	100000	10000

Atribuição de Estado

- Implícito:

```
type mc_state_type is (idle, read1, read2, read3, read4, write);
attribute enum_encoding: string;
attribute enum_encoding of mc_state_type :
          type is "0000 0100 1000 1001 1010 1011";
```

- Explícito: usando *std_logic_vector*
- *Listing 10.4*: Arquitetura `state_assign_arch` em list_10_01_to_06_memctrl.vhd

```
211 architecture state_assign_arch of mem_ctrl is
212   constant idle:  std_logic_vector(3 downto 0):="0000";
213   constant write: std_logic_vector(3 downto 0):="0100";
214   constant read1: std_logic_vector(3 downto 0):="1000";
215   constant read2: std_logic_vector(3 downto 0):="1001";
216   constant read3: std_logic_vector(3 downto 0):="1010";
217   constant read4: std_logic_vector(3 downto 0):="1011";
218   signal state_reg,state_next: std_logic_vector(3 downto 0);
219 begin
```


Lidando com os estados não utilizados

- Muitas representações binárias não são usadas
- O que acontece se o FSM entrar em um estado não utilizado?
  - Ignorar a condição
  - FSM seguro (tolerante a falhas): chega a um estado de erro ou retorna ao estado inicial.
- Fácil para a atribuição explícita de estado.

Fim!